

SOLID Principle Mapping

Movie Ticket Booking System

Course: B. Tech CSE

Semester: 5th

Subject: System Design

Subject Code: TCS-504

Assignment: Assignment 1

Single Responsibility Principle (SRP)

Principle:

A class should have one primary responsibility and should have only one reason to change.

Class	Responsibility
Movie	Stores movie information such as title, language and duration.
Seat	Represents one physical seat and its seat type.
Show	Represents one movie screening on a particular screen and time.
ShowSeat	Maintains the availability status of one seat for one show.
Booking	Stores booking information and manages booking status.
PriceCalculator	Calculates the total booking price.
Payment	Defines the payment contract.
TicketPrinter	Formats and prints the ticket.
BookingService	Coordinates the complete booking workflow.

Example:

Booking does not calculate prices or print tickets. Price calculation is handled by PriceCalculator, while ticket printing is handled by TicketPrinter.

Therefore, a pricing change affects PriceCalculator rather than the Booking class.

Open/Closed Principle (OCP)

Principle:

Software entities should be open for extension but closed for modification.

Application in the system:

The payment system uses an abstract Payment class:

```
class Payment {  
  
public:  
  
    virtual ~Payment() = default;  
  
    virtual bool pay(double amount) = 0;  
  
};
```

The existing payment methods implement this abstraction:

Payment

```
|  
+-- UpiPayment  
+-- CardPayment  
+-- CashPayment
```

If a new payment method such as NetBankingPayment is required, a new class can inherit from Payment without changing the existing payment implementations or the core booking workflow.

Example:

```
class NetBankingPayment : public Payment {  
public:  
    bool pay(double amount) override {  
        // Net banking implementation  
    }  
};
```

This follows the assignment's requirement that adding a new payment method should not require modifying the existing booking workflow.

Liskov Substitution Principle (LSP)

Principle:

Objects of a derived class should be usable wherever objects of the base class are expected without breaking the program.

Application in the system:

UpiPayment, CardPayment, and CashPayment inherit from the abstract Payment class.

Payment*

```
|  
+-- UpiPayment  
+-- CardPayment  
+-- CashPayment
```

BookingService accepts a Payment reference/pointer and invokes:

```
payment.pay(amount);
```

It does not need to know which specific payment implementation is being used.

Therefore, all three payment classes can be substituted for Payment.

This matches the assignment's LSP requirement that every payment child should work through a Payment* without additional setup.

Dependency Inversion Principle (DIP)

Principle:

High-level modules should depend on abstractions rather than concrete implementations.

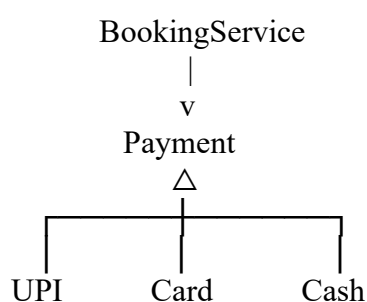
Application in the system:

BookingService does not create a specific payment implementation such as:

```
new UpiPayment();
```

Instead, the required payment object is supplied to the booking operation through the Payment abstraction.

Conceptually:



This allows BookingService to work with different payment methods without being tightly coupled to a particular payment class.

The assignment specifically asks whether BookingService should create a CardPayment itself or receive a Payment&; the implemented design follows the abstraction-based approach.

SOLID Summary

SOLID Principle	Applied Where	Benefit
SRP	Booking, PriceCalculator, TicketPrinter, Payment, etc.	Each class has one clear responsibility.
OCP	Payment hierarchy	New payment methods can be added without modifying the booking workflow.

SOLID Principle	Applied Where	Benefit
LSP	UpiPayment, CardPayment, CashPayment	All payment implementations can substitute the Payment abstraction.
DIP	BookingService → Payment	Booking workflow depends on the payment abstraction instead of a concrete payment type.

One Principle Deliberately NOT Followed

Interface Segregation Principle (ISP) — Deliberately Not Applicable

Principle:

Clients should not be forced to depend on methods they do not use.

In this project, Payment contains only the single essential operation:

```
virtual bool pay(double amount) = 0;
```

There is no unnecessary large interface containing unrelated operations such as:

```
refund()
```

```
validateCard()
```

```
generateReceipt()
```

```
savePayment()
```

```
sendSMS()
```

Because the payment abstraction is already very small, introducing multiple interfaces only to demonstrate ISP would add unnecessary complexity to this small assignment.

Therefore, **ISP is deliberately not demonstrated as a separate design feature**, keeping the implementation within the scope of the assignment.

Conclusion

The Movie Ticket Booking System applies **SRP, OCP, LSP and DIP** through separation of responsibilities and the abstract payment hierarchy. The design allows payment methods to be extended without modifying the central booking workflow, while individual classes remain focused on their designated responsibilities.